

ЛАБОРАТОРНАЯ РАБОТА №8

ОБРАБОТКА СОБЫТИЙ МЫШИ

Цель работы: Получение практических навыков обработки событий
Программные средства: Eclipse IDE с SDK JavaSE

7.1. Теоретическая справка

Обработка событий (event handling) — это механизм, позволяющий двум или более объектам системы обмениваться информацией об изменении состояния. В системах, управляемых событиями, один объект всегда выступает в качестве генератора событий, создавая объекты событий, предназначенные для обозначения определенных изменений в состоянии этого объекта. Затем генератор событий передает событие одному или нескольким зарегистрированным получателям, вызывая предназначенный для этого метод каждого объекта-получателя.

Программно обработка событий реализуется следующим образом. Для каждого типа события создается соответствующий класс вида xxxxxxEvent. Источник события (класс, который может генерировать экземпляры данного класса при возникновении заданного события), такой, например, как класс Component, хранит список зарегистрировавшихся у него обработчиков событий. При возникновении события источник создает экземпляр соответствующего класса и передает его тому обработчику, который должен обработать это событие. Для того чтобы найти нужный обработчик, источник вызывает у всех известных ему обработчиков специальный метод, передавая этому методу экземпляр класса события в качестве параметра.

Обработчики событий — это интерфейсы, в которых содержатся определения методов, вызываемых источниками событий при возникновении определенных обстоятельств: щелчок на кнопке, перемещение мыши и т.п. В приложении Б представлены интерфейсы широко используемых обработчиков событий.

Переопределив метод mouseDown, можно отслеживать нажатия клавиши мыши. Прототип этого метода приведен ниже:

```
public boolean mouseDown(Event evt, int x, int y);
```

Через параметр evt методу передается ссылка на объект Event, с помощью которой метод может получить полную информацию о событии.

Анализируя содержимое параметров x и y, приложение может определить координаты курсора на момент возникновения события.

Следует обратить внимание, что некоторые события составные, так для отслеживания двойного щелчка мыши не предусмотрено никакого отдельного метода. Однако анализируя содержимое поля clickCount переменной evt, можно определить кратность щелчка мыши:

```
if(evt.clickCount > 1)    // Двойной щелчок
    showStatus("Mouse Double Click");
else    // Одинарный щелчок
    showStatus("Mouse Down");
```

При отпускании клавиши мыши управление получает метод mouseUp:

```
public boolean mouseUp(Event evt, int x, int y);
```

Анализируя параметры x и y можно определить координаты точки, в которой пользователь отпустил клавишу мыши.

Когда пользователь перемещает курсор мыши над элементом управления, в процессе перемещения происходит вызов метода mouseMove:

```
public boolean mouseMove(Event evt, int x, int y);
```

Через переменные x и y передаются текущие координаты курсора мыши.

Операция Drag and Drop выполняется следующим образом: пользователь нажимает клавишу мыши и, не отпуская ее, начинает перемещать курсор мыши. При этом происходит вызов метода mouseDrag:

```
public boolean mouseDrag(Event evt, int x, int y);
```

в котором через переменные x и y передаются текущие координаты курсора мыши.

Метод mouseEnter получает управление, когда курсор мыши в процессе перемещения по экрану попадает в область элемента управления:

```
public boolean mouseEnter(Event evt, int x, int y);
```

Этот метод можно использовать для активизации элемента управления, на который указывает курсор мыши.

Метод mouseExit вызывается при покидании курсором элемента управления:

```
public boolean mouseExit(Event evt, int x, int y);
```

Если пользователь убрал курсор из элемента управления, активизированного методом mouseEnter, то метод mouseExit может переключить этот элемент управления в пассивное состояние.

При обработке событий важна информация о месте, компоненте и других особенностях произошедшего события. Для этого следует использовать получаемый при возникновении события объект класса Event.

Таблица – Атрибуты класса Event

Тип	Название	Описание
Object	target	Ссылка на компонент, который первоначально получил сообщение
long	when	Момент времени в который произошло событие
int	id	Тип события*.
int	x, y	Координата x, y точки в которой произошло действие, относительно координат компоненты которая обрабатывает событие.
int	key	Для событий клавиатуры — код нажатой клавиши
int	modifiers	Комбинация значений SHIFT_MASK, CTRL_MASK, META_MASK и ALT_MASK для клавиш shift, control, meta и alt
int	clickCount	Количество последовательных нажатий мыши (используется в MOUSE_DOWN).
Object	arg	Аргумент зависящий от события (для объектов Button

		представляет собой объект типа String, который содержит надпись на этой кнопке)
Event	evt	Следующее событие в связанном списке событий.

Обычно экземпляр класса Event создается исполняющей системой Java, для доступа к свойствам, следует использовать методы:

int getClickCount() – возвращает число щелчков мышью, связанных с этим событием;

Point getLocationOnScreen() – возвращает абсолютную позицию x, y на экране, где произошло события;

Point getPoint() – возвращает x, y позиции события относительно исходного компонента;

int getX(), int getY() – возвращают горизонталь x и вертикаль y позиции события относительно исходного компонента;

String paramString() – возвращает строку параметра, идентифицирующую это событие;

public Component getComponent() – возвращает инициатора события;

public String toString() – возвращает строковое представление этого объекта.

7.2 Задание и рекомендацию по его выполнению

Задача 1. Реализуйте и исследуйте программу, которая сообщает о действии, выполняемом мышкой на компоненте.

В приложении с кнопкой реализуйте интерфейс MouseListener, добавьте внизу текстовую метку eventLabel для вывода действий мыши и изучите, какие виды событий происходят при нажатии.

```
public class XXXX implements MouseListener {
    //нажатие кнопки мыши
    public void mouseClicked(MouseEvent e) {
        JButton button = (JButton) e.getSource();
        String text = "<html><b>" + button.getText()
            + "      mouseReleased()      <br>" +
        button.getText()
            + " mouseClicked() </b><html>";
        eventLabel.setText(text);
    }
    //появление курсора мыши в компоненте
    public void mouseEntered(MouseEvent e) {
        JButton button = (JButton) e.getSource();
        eventLabel.setText(button.getText() + "
mouseEntered() ");
    }
    //выход курсора мыши из компонента
    public void mouseExited(MouseEvent e) {
        JButton button = (JButton) e.getSource();
        eventLabel.setText(button.getText() + "
mouseExited() ");
    }
    //нажатие кнопки мыши
```

```

        public void mousePressed(MouseEvent e) {
            JButton button = (JButton) e.getSource();
            eventLabel.setText(button.getText() + "
mousePressed()");
        }
        //отпускание кнопки мыши
        public void mouseReleased(MouseEvent e) {
            JButton button = (JButton) e.getSource();
            eventLabel.setText(button.getText() + "
mouseReleased()");
        }
    }

```

Обратите внимание, что слушатели должны быть добавлены к объекту, для работы:

```
this.addMouseListener(this);
```

Сначала подключите обработчик к кнопке, потом текстовой метке, затем панели-контейнеру. Изучите особенности работы программы. Внесите изменения в код, чтобы сообщалось по какому компоненту окну в данный момент проходит мышь.

Задача 2. Написать программу, которая позволяет рисовать линии, следуя движениям мыши.

Используйте операцию переноса объектов – пользователь нажимает клавишу мыши, перемещает курсор и затем отпускает клавишу мыши. При движении мыши рисуется тонкая серая линия, которая заменяется линией с заранее выбранным цветом после отпускания клавиши мыши. Для этого реализуйте для панели, на которой происходит рисование интерфейс `MouseMotionListener` и метод `paint()` с прорисовкой запомненного полигона.

7.3 Примеры обработчиков событий

Пример 1. Программа, позволяющая рисовать на панели, выбранным с помощью стандартного диалога цветом.

Интерфейс состоит из панели для рисования (`JPanel panel`), кнопки выбора цвета `button` (`JButton("Выбор цвета")`) и текстовой метки `eventLabel` (`JLabel("Действия мыши")`).

В классе приложения задействованы переменные:

`Color col` – для хранения выбранного цвета рисования;

`int x, y` – для сохранения предыдущей позиции мыши.

Реализация интерфейса `MouseMotionListener`:

```

public void mouseDragged(MouseEvent arg0) {
    eventLabel.setText("Moved");
    panel.getGraphics();
    g.setColor(col);
    g.drawLine(x, y, arg0.getX(), arg0.getY()
    }

    public void mouseMoved(MouseEvent arg0) {
        eventLabel.setText("Dragged");
        x = arg0.getX();
    }

```

```

        y = arg0.getY();
    }

```

Обработчик движения мыши добавляется к панели:

```

panel.addMouseListener(this);

```

Анонимный класс-адаптер для обработки нажатия на кнопку может быть реализован следующим образом:

```

public void mouseClicked(MouseEvent arg0) {
    col = JColorChooser.showDialog(panel, "Цвет",
col); //выбор цвета из палитры
}

```

ПРИЛОЖЕНИЕ В. ОСНОВНЫЕ СОБЫТИЯ AWT

Класс события	Интерфейс слушателя	Элемент	Методы слушателя	Значение
ActionEvent	ActionListener	Button List MenuItem TextField	actionPerformed()	Нажатие кнопки/ Двойной щелчок /Выбор пункта меню/ Конец редактирования (Enter)
AdjustmentEvent	AdjustmentListener	Scrollbar	adjustmentValueChanged()	Реализация прокрутки
ComponentEvent	ComponentListener	Component	componentHidden() componentMoved() componentResized() componentShown()	Перемещение, изменение размеров, скрытие/видимость
ContainerEvent	ContainerListener	Container	componentAdded() componentRemoved()	Добавление /удаление из контейнера
FocusEvent	FocusListener	Component	focusGained() focusLost()	Получение/потеря фокуса
ItemEvent	ItemListener	Checkbox List	itemStateChanged()	Установка/сброс флага
KeyEvent	KeyListener	Component	keyPressed() keyReleased() keyTyped()	Нажатие/отпускание клавиши
MouseEvent	MouseListener	Component	mouseClicked() mouseEntered() mouseExited() mousePressed() mouseReleased()	Нажатие/отпускание мыши, вошел/покинул область элемента,
	MouseMotionListener	Component	mouseDragged() mouseMoved()	Перемещение мыши без/с учетом нажатой кнопки
TextEvent	TextListener	TextComponent	textValueChanged()	Изменения в тексте элемента

